

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное обеспечение

Дисциплина «Алгоритмы и структуры данных»

Отчёт по блоку задач №4

Выполнил
Линейский Аким Евгеньевич
Р3215
Преподаватель
Тараканов Денис Сергеевич

Санкт-Петербург, 2026

1. Задача М. Гусеница

1.1. Условие задачи

М. Цивилизация

✓ Полное решение

Ограничение времени	1.5 секунд
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Карта мира в компьютерной игре «Цивилизация» версии 1 представляет собой прямоугольник, разбитый на квадратики. Каждый квадратик может иметь один из нескольких возможных рельефов, для простоты ограничимся тремя видами рельефов — поле, лес и вода. Поселенец перемещается по карте, при этом на перемещение в клетку, занятую полем, необходима одна единица времени, на перемещение в лес — две единицы времени, а перемещаться в клетку с водой нельзя.

У вас есть один поселенец, вы определили место, где нужно построить город, чтобы как можно скорее завладеть всем миром. Найдите маршрут переселенца, приводящий его в место строительства города, требующий минимального времени. На каждом ходе переселенец может перемещаться в клетку, имеющую общую сторону с той клеткой, где он сейчас находится.

Формат ввода

Во входном файле записаны два натуральных числа N и M , не превосходящих 1000 — размеры карты мира (N — число строк в карте, M — число столбцов). Затем заданы координаты начального положения поселенца x и y , где x — номер строки, y — номер столбца на карте ($1 \leq x \leq N, 1 \leq y \leq M$), строки нумеруются сверху вниз, столбцы — слева направо. Затем аналогично задаются координаты клетки, куда необходимо привести поселенца.

Далее идет описание карты мира в виде N строк, каждая из которых содержит M символов. Каждый символ может быть либо «.» (точка), обозначающим поле, либо «W», обозначающим лес, либо «#», обозначающим воду. Гарантируется, что начальная и конечная клетки пути переселенца не являются водой.

Формат вывода

В первой строке выходного файла выведите количество единиц времени, необходимое для перемещения поселенца (перемещение в клетку с полем занимает 1 единицу времени, перемещение в клетку с лесом — 2 единицы времени). Во второй строке выходного файла выведите последовательность символов, задающих маршрут переселенца. Каждый символ должен быть одним из четырех следующих: «N» (движение вверх), «E» (движение вправо), «S» (движение вниз), «W» (движение влево). Если таких маршрутов несколько, выведите любой из них.

Если дойти из начальной клетки в конечную невозможно, выведите число -1 .

1.2. Идея решения

Задача поиска кратчайшего пути на клетчатом поле с разными стоимостями перемещения сводится к нахождению кратчайших путей в графе с неотрицательными весами ребер. Для ее решения используется алгоритм Дейкстры с приоритетной очередью.

Клетки поля выступают в качестве вершин графа, а переходы в четыре соседние стороны образуют ребра. Перемещение в обычную пустую клетку имеет вес один, а проход через

воду требует затрат в два очка. Стеновые ограждения и пустые пространства за границами карты игнорируются при генерации переходов. Алгоритм Дейкстры последовательно извлекает вершины с минимальным текущим расстоянием из кучи `std::priority_queue` и обновляет метки соседей. Для восстановления пути ведется матрица предков, куда сохраняются координаты предыдущей точки и направление движения. После достижения финиша траектория восстанавливается в обратном порядке от конечной точки к начальной.

1.3. Исходный код программы (C++)

```
1 #include <algorithm>
2 #include <iostream>
3 #include <queue>
4 #include <string>
5 #include <vector>
6
7 using namespace std;
8
9 struct Cell {
10     int dist;
11     int x, y;
12
13     bool operator>(const Cell& other) const {
14         return dist > other.dist;
15     }
16 };
17
18 struct Point {
19     int x, y;
20     char dir;
21 };
22
23 const int dx[] = {-1, 0, 1, 0};
24 const int dy[] = {0, 1, 0, -1};
25 const char dir_chars[] = {'N', 'E', 'S', 'W'};
26
27 int main() {
28     ios_base::sync_with_stdio(false);
29     cin.tie(NULL);
30
31     int n, m;
32     if (!(cin >> n >> m))
33         return 0;
34
35     int start_x, start_y, end_x, end_y;
36     cin >> start_x >> start_y >> end_x >> end_y;
37
38     start_x--;
39     start_y--;
40     end_x--;
41     end_y--;
42
43     vector<string> grid(n);
44     for (int i = 0; i < n; ++i) {
45         cin >> grid[i];
46     }
47
48     const int INF = 1e9;
49     vector<vector<int>> dist(n, vector<int>(m, INF));
```

```

50 vector<vector<Point>> parent(n, vector<Point>(m, {-1, -1, ' '}));
51
52 priority_queue<Cell, vector<Cell>, greater<Cell>> pq;
53
54 dist[start_x][start_y] = 0;
55 pq.push({0, start_x, start_y});
56
57 while (!pq.empty()) {
58     Cell curr = pq.top();
59     pq.pop();
60
61     int x = curr.x;
62     int y = curr.y;
63     int d = curr.dist;
64
65     if (d > dist[x][y])
66         continue;
67
68     if (x == end_x && y == end_y)
69         break;
70
71     for (int i = 0; i < 4; ++i) {
72         int nx = x + dx[i];
73         int ny = y + dy[i];
74
75         if (nx >= 0 && nx < n && ny >= 0 && ny < m && grid[nx][ny] != '#' &&
76             grid[nx][ny] != ' ') {
77             int weight = (grid[nx][ny] == 'W') ? 2 : 1;
78
79             if (dist[x][y] + weight < dist[nx][ny]) {
80                 dist[nx][ny] = dist[x][y] + weight;
81                 parent[nx][ny] = {x, y, dir_chars[i]};
82                 pq.push({dist[nx][ny], nx, ny});
83             }
84         }
85     }
86
87     if (dist[end_x][end_y] == INF) {
88         cout << -1 << "\n";
89     } else {
90         cout << dist[end_x][end_y] << "\n";
91         string path = "";
92         int cx = end_x;
93         int cy = end_y;
94
95         while (cx != start_x || cy != start_y) {
96             path += parent[cx][cy].dir;
97             int px = parent[cx][cy].x;
98             int py = parent[cx][cy].y;
99             cx = px;
100            cy = py;
101        }
102
103        reverse(path.begin(), path.end());
104        cout << path << "\n";
105    }
106
107    return 0;
108 }

```

2. Задача N. Свинки-копилки

2.1. Условие задачи

N. Свинки-копилки

✓ Полное решение

Ограничение времени	1 секунда
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

У Васи есть n свинок-копилков, свинки занумерованы числами от 1 до n . Каждая копилка может быть открыта единственным соответствующим ей ключом или разбита.

Вася положил ключи в некоторые из копилков (он помнит, какой ключ лежит в какой из копилков). Теперь Вася собрался купить машину, а для этого ему нужно достать деньги из всех копилков. При этом он хочет разбить как можно меньшее количество копилков (ведь ему еще нужно копить деньги на квартиру, дачу, вертолет...). Помогите Васе определить, какое минимальное количество копилков нужно разбить.

Формат ввода

В первой строке содержится число n — количество свинок-копилков ($1 \leq n \leq 100$). Далее идет n строк с описанием того, где лежит ключ от какой копилки: в i -й из этих строк записан номер копилки, в которой находится ключ от i -й копилки.

Формат вывода

Выведите единственное число: минимальное количество копилков, которые необходимо разбить.

Пример

Ввод

4
2
1
2
4

Вывод

2

[Скачать условие задачи](#)

2.2. Идея решения

Ситуация с ключами и свиньями моделируется в виде ориентированного графа, где каждая копилка является вершиной, а от нее отходит ровно одно ребро к той копилке, внутри которой спрятан ключ. Поскольку полустепень исхода каждой вершины строго равна единице, граф представляет собой функциональный граф и распадается на несколько компонент связности, каждая из которых содержит ровно один цикл.

Чтобы разбить минимальное число свинок, достаточно разбить ровно по одной копилке в каждом изолированном цикле. Поиск циклов реализуется обходом в глубину. Массив состояний `visited` хранит три метки: ноль означает не посещенную вершину, единица

отмечает нахождение в текущем стеке рекурсии, а двойка сигнализирует о полном завершении обработки. Если во время обхода алгоритм наткнется на вершину со значением один, это означает обнаружение нового цикла, и общий счетчик увеличивается на единицу.

2.3. Исходный код программы (C++)

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 void dfs(int v, const vector<int> &next_pig, vector<int> &visited,
7         int &cycles) {
8     visited[v] = 1;
9
10    int to = next_pig[v];
11    if (visited[to] == 0) {
12        dfs(to, next_pig, visited, cycles);
13    } else if (visited[to] == 1) {
14        cycles++;
15    }
16
17    visited[v] = 2;
18 }
19
20 int main() {
21     ios_base::sync_with_stdio(false);
22     cin.tie(NULL);
23
24     int n;
25     if (!(cin >> n))
26         return 0;
27
28     vector<int> next_pig(n);
29     for (int i = 0; i < n; ++i) {
30         cin >> next_pig[i];
31         next_pig[i]--;
32     }
33
34     vector<int> visited(n, 0);
35     int cycles = 0;
36
37     for (int i = 0; i < n; ++i) {
38         if (visited[i] == 0) {
39             dfs(i, next_pig, visited, cycles);
40         }
41     }
42
43     cout << cycles << "\n";
44
45     return 0;
46 }
```

3. Задача О. Откуда берутся пушки?

3.1. Условие задачи

О. Долой списывание!

✓ Полное решение

Ограничение времени	2 секунды
Ограничение памяти	64 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Во время теста Михаил Дмитриевич заметил, что некоторые лкшата обмениваются записками. Сначала он хотел поставить им всем двойки, но в тот день Михаил Дмитриевич был добрым, а потому решил разделить лкшат на две группы: списывающих и дающих списывать, и поставить двойки только первым.

У Михаила Дмитриевича записаны все пары лкшат, обменивавшихся записками. Требуется определить, сможет ли он разделить лкшат на две группы так, чтобы любой обмен записками осуществлялся от лкшонка одной группы лкшонку другой группы.

Формат ввода

В первой строке находятся два числа N и M — количество лкшат и количество пар лкшат, обменивающихся записками ($1 \leq N \leq 100, 0 \leq M \leq \frac{N(N-1)}{2}$). Далее в M строках расположены описания пар лкшат: два различных числа, соответствующие номерам лкшат, обменивающихся записками (нумерация лкшат идёт с 1). Каждая пара лкшат перечислена не более одного раза.

Формат вывода

Необходимо вывести ответ на задачу Павла Олеговича. Если возможно разделить лкшат на две группы, выведите «YES»; иначе выведите «NO».

Пример

Ввод 

```
3 2
1 2
2 3
```

Вывод 

```
YES
```

[Скачать условие задачи](#)

3.2. Идея решения

Требование разделить студентов на две бесконфликтные группы проверяет граф на двудольность. В этой модели вершины соответствуют учащимся, а неориентированные ребра связывают пары студентов, заявивших о взаимной неприязни.

Проверка графа на двудольность выполняется с помощью раскраски вершин в два альтернативных цвета посредством поиска в глубину. При входе в очередную нераскрашенную вершину ей присваивается базовый цвет. Все смежные с ней вершины в рамках рекурсивных вызовов должны получить противоположный цвет. Если в процессе обхода обнаруживается соседняя вершина, уже окрашенная в тот же цвет, что и текущая, то граф

содержит цикл нечетной длины. В такой ситуации корректное разбиение на две группы становится невозможным. Если вся проверка завершается успешно, выводится подтверждающий ответ.

3.3. Исходный код программы (C++)

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 bool dfs(int v, int c, const vector<vector<int>> &adj, vector<int> &color) {
7     color[v] = c;
8
9     for (int to : adj[v]) {
10         if (color[to] == 0) {
11             int next_color = (c == 1) ? 2 : 1;
12             if (!dfs(to, next_color, adj, color)) {
13                 return false;
14             }
15         } else if (color[to] == color[v]) {
16             return false;
17         }
18     }
19     return true;
20 }
21
22 int main() {
23     ios_base::sync_with_stdio(false);
24     cin.tie(NULL);
25
26     int n, m;
27     if (!(cin >> n >> m))
28         return 0;
29
30     vector<vector<int>> adj(n);
31     for (int i = 0; i < m; ++i) {
32         int u, v;
33         cin >> u >> v;
34         u--;
35         v--;
36         adj[u].push_back(v);
37         adj[v].push_back(u);
38     }
39
40     vector<int> color(n, 0);
41     bool possible = true;
42
43     for (int i = 0; i < n; ++i) {
44         if (color[i] == 0) {
45             if (!dfs(i, 1, adj, color)) {
46                 possible = false;
47                 break;
48             }
49         }
50     }
51
52     if (possible) {
53         cout << "YES\n";
```



```
54 } else {  
55     cout << "NO\n";  
56 }  
57  
58 return 0;  
59 }
```

4. Задача Р. Авиаперелеты

4.1. Условие задачи

Р. Авиаперелёты

✓ Полное решение

Ограничение времени	2 секунды
Ограничение памяти	256 Мб
Ввод	стандартный ввод или avia.in
Вывод	стандартный вывод или avia.out

Главного конструктора Петю попросили разработать новую модель самолёта для компании «Air Бубундия». Оказалось, что самая сложная часть заключается в подборе оптимального размера топливного бака.

Главный картограф «Air Бубундия» Вася составил подробную карту Бубундии. На этой карте он отметил расход топлива для перелёта между каждой парой городов.

Петя хочет сделать размер бака минимально возможным, для которого самолёт сможет долететь от любого города в любой другой (возможно, с дозаправками в пути).

Формат ввода

Первая строка входного файла содержит натуральное число n ($1 \leq n \leq 1\,000$) — число городов в Бубундии. Далее идут n строк по n чисел каждая. j -е число в i -й строке равно расходу топлива при перелёте из i -го города в j -й. Все числа не меньше нуля и меньше 10^9 . Гарантируется, что для любого i в i -й строчке i -е число равно нулю.

Формат вывода

Первая строка выходного файла должна содержать одно число — оптимальный размер бака.

Пример

Ввод 	Вывод 
4 0 10 12 16 11 0 8 9 10 13 0 22 13 10 17 0	10

[Скачать условие задачи](#)

4.2. Идея решения

Для нахождения минимальной вместимости бака, позволяющей добраться из любого города в любой другой, применяется бинарный поиск по ответу в сочетании с проверкой графа на сильную связность. Нижняя граница поиска равна нулю, а верхняя соответствует максимальному значению стоимости дорог в матрице.

На каждом шаге бинарного поиска выбирается среднее значение веса бака. При таком ограничении граф становится подграфом, содержащим только те ориентированные ребра, вес которых не превышает выбранный лимит. Проверка сильной связности полученного подграфа базируется на свойстве вершины номер ноль. Сначала запускается обход в глу-

бину `dfs1` из нулевой вершины по исходным ребрам. Если удалось посетить все вершины, то из нуля достижимы все города. Затем запускается второй обход `dfs2` из нулевой вершины, но уже по транспонированным ребрам, что эквивалентно проверке достижимости нуля из всех остальных точек. Если оба условия выполнены, подграф сильно связан, и мы пробуем уменьшить бак.

4.3. Исходный код программы (C++)

```

1 #include <algorithm>
2 #include <fstream>
3 #include <vector>
4
5 using namespace std;
6
7 void dfs1(int v, int max_w, const vector<vector<int>> &adj,
8         vector<bool> &visited, int n) {
9     visited[v] = true;
10    for (int to = 0; to < n; ++to) {
11        if (!visited[to] && adj[v][to] <= max_w) {
12            dfs1(to, max_w, adj, visited, n);
13        }
14    }
15 }
16
17 void dfs2(int v, int max_w, const vector<vector<int>> &adj,
18         vector<bool> &visited, int n) {
19     visited[v] = true;
20    for (int to = 0; to < n; ++to) {
21        if (!visited[to] && adj[to][v] <= max_w) {
22            dfs2(to, max_w, adj, visited, n);
23        }
24    }
25 }
26
27 bool check(int max_w, const vector<vector<int>> &adj, int n) {
28     vector<bool> visited(n, false);
29
30     dfs1(0, max_w, adj, visited, n);
31     for (int i = 0; i < n; ++i) {
32         if (!visited[i])
33             return false;
34     }
35
36     fill(visited.begin(), visited.end(), false);
37     dfs2(0, max_w, adj, visited, n);
38     for (int i = 0; i < n; ++i) {
39         if (!visited[i])
40             return false;
41     }
42
43     return true;
44 }
45
46 int main() {
47     ifstream cin("input.txt");
48     ofstream cout("output.txt");
49
50     int n;
51     if (!(cin >> n))

```

```

52     return 0;
53
54     vector<vector<int>> adj(n, vector<int>(n));
55     int max_edge = 0;
56     for (int i = 0; i < n; ++i) {
57         for (int j = 0; j < n; ++j) {
58             cin >> adj[i][j];
59             max_edge = max(max_edge, adj[i][j]);
60         }
61     }
62
63     int left = 0;
64     int right = max_edge;
65     int ans = max_edge;
66
67     while (left <= right) {
68         int mid = left + (right - left) / 2;
69         if (check(mid, adj, n)) {
70             ans = mid;
71             right = mid - 1;
72         } else {
73             left = mid + 1;
74         }
75     }
76
77     cout << ans << "\n";
78
79     return 0;
80 }

```